Monte Carlo vs. Sarsa(λ) on Text Flappy Bird

Côme Genet

Abstract. We apply on-policy first-visit Monte Carlo (MC) control and tabular Sarsa(λ) with replacing eligibility traces to `TextFlappyBird-v0` [2]. The continuous (d_x, d_y) observation is discretized into a 20×20 grid ($|\mathcal{S}| = 400$). After 6 000 episodes, Sarsa(λ) reaches 492.1 ± 349.8 pipes in greedy evaluation against 95.3 ± 95.6 for MC. We report parameter sweeps, value-function analysis, and a cross-configuration generalization experiment.

1 Setup

Environment. `TextFlappyBird-v0` [2] returns (d_x, d_y), the distance of the player from the centre of the closest pipe gap. Actions are binary (0: idle, 1: flap); reward is $+1$ per step survived, -1 on collision. Default configuration: height $= 15$, width $= 20$, pipe gap $= 4$. We discretize each axis into 20 uniform bins, giving a Q-table of $400 \times 2 = 800$ entries. The RNG is seeded once at instantiation; each episode draws a distinct pipe sequence.

Agent 1 — MC. On-policy first-visit MC control [1, Alg. 5.4]. Returns $G_t = \sum_k \gamma^k r_{t+k+1}$ are accumulated backwards; $Q(s, a)$ is updated as a running average over first visits. Policy: ε-greedy, exponential decay ($\varepsilon_0 = 1$, $\varepsilon_\infty = 0.01$, rate $= 0.9985$), $\gamma = 0.99$.

Agent 2 — Sarsa(λ). Tabular Sarsa(λ) with *replacing* traces [1, Sec. 12.7]. At each step:

$$\delta_t = r_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t), \quad e(S_t, A_t) \leftarrow 1,$$
$$Q \leftarrow Q + \alpha \delta_t e, \quad e \leftarrow \gamma \lambda\, e.$$

On terminal steps the bootstrap term is dropped and traces are reset. Replacing traces cap e at 1 on revisit, preventing trace explosion. Hyperparameters: $\alpha = 0.1$, $\lambda = 0.8$, $\gamma = 0.99$; same ε-schedule as MC.

2 Results

Learning curves. Fig. 1 shows the 100-episode moving average over 6 000 episodes. Sarsa(λ) accelerates sharply once $\varepsilon < 0.05$ (episode $\approx 2\,500$), stabilising around 75–83 pipes; MC plateaus at 20–25 pipes, limited by the high variance of long-episode returns and delayed credit assignment.

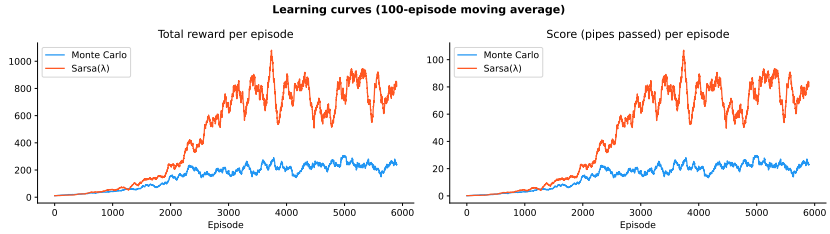

Fig. 1. 100-episode moving-average reward (left) and score (right). Sarsa(λ) converges faster and to a higher asymptote.

Value functions. Fig. 2 shows $V(s) = \max_a Q(s,a)$ after training. The MC value function presents a smooth gradient centred on the gap but retains low values in rarely visited states, consistent with its high evaluation variance. Sarsa(λ) assigns systematically higher values across the central region, reflecting more efficient credit propagation through eligibility traces.

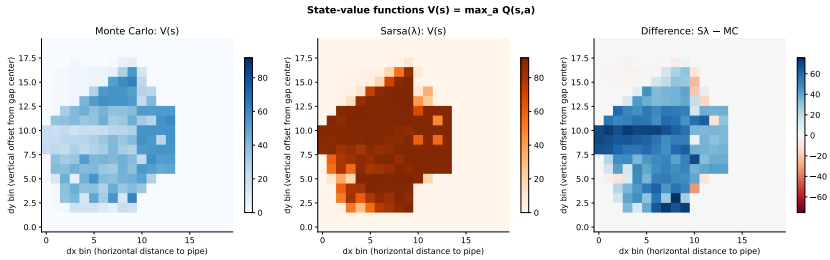

Fig. 2. $V(s) = \max_a Q(s,a)$ for MC (left) and Sarsa(λ) (centre); difference $V_{\mathrm{S}\lambda} - V_{\mathrm{MC}}$ (right).

Greedy evaluation & parameter sweeps. Table 1 reports 200 greedy episodes ($\varepsilon = 0$, `max_steps`$= 10\,000$). Sarsa(λ) achieves a $5\times$ higher mean score; variance remains large for both agents as tabular methods cannot interpolate to unseen pipe configurations. Note that both agents significantly outperform a random policy (1–2 pipes); the high variance reflects configuration-dependent performance rather than poor learning. Fig. 3 shows the ε-decay sweep: both agents collapse when decay is too slow (≥ 0.998), with an optimal range of 0.992–0.995. Additional sweeps over $\lambda \in \{0, \ldots, 0.95\}$ and $\alpha \in \{0.01, \ldots, 0.4\}$ (not shown) identify $\lambda = 0.4$ and $\alpha = 0.05$ as optimal.

Table 1. Greedy evaluation (200 episodes, mean $\pm$ std).

Agent	Update	Bootstrap	Score	Reward
MC	Episode end	No	95.3 ± 95.6	960.6 ± 955.4
Sarsa(λ)	Per step	Yes	492.1 ± 349.8	4933.7 ± 3497.4

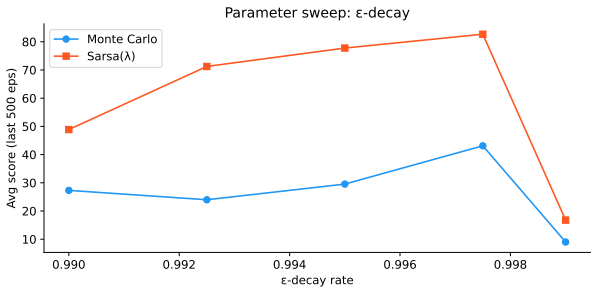

Fig. 3. Avg. score vs. ε-decay rate (3 000-episode runs).

3 Discussion

MC vs. Sarsa(λ). The key difference is *update timing*: MC uses the full unbiased return but suffers high variance on long episodes; Sarsa(λ) bootstraps at each step, reducing variance at the cost of bias. In TFB, where a good policy survives thousands of steps, this variance reduction is decisive. Eligibility traces further accelerate credit assignment by distributing the TD error across recently visited state-action pairs. MC is simpler (only ε, γ); Sarsa(λ) adds α and λ but the 5× performance gain justifies the extra tuning.

Environment versions. `TextFlappyBird-v0` returns a compact (d_x, d_y) vector amenable to 20×20 discretization (800 Q-values, convergence in ∼3 000 episodes). `TextFlappyBird-screen-v0` returns the full $H \times W$ character screen; treating cells as binary gives 2^{300} states for the default config, making tabular methods entirely intractable. The screen version implicitly encodes multiple upcoming pipes (richer information) but is incompatible with the agents studied here.

Original Flappy Bird gym. `flappy-bird-gym` [3] exposes either a $288 \times 512 \times 3$ RGB frame or a continuous feature vector, both too high-dimensional for tabular RL. Sarsa(λ) extends naturally to the semi-gradient setting ($\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta_t \mathbf{z}_t$, [1, Sec. 12.9]); the pixel case would require a CNN encoder with DQN [4]. MC does not extend as cleanly without online updates and explicit variance reduction.

Cross-configuration generalization. Agents trained on config A (H= 15, W= 20, gap= 4) evaluated on config B (H= 20, W= 25, gap= 3) score 0.33 ± 0.84 (MC) and 1.92 ± 2.54 (Sarsa(λ)) pipes, versus 72.66 ± 74.90 for a Sarsa(λ) trained directly on B. Transfer fails because bin indices are geometry-specific: the same physical (d_x, d_y) maps to a different bin in B, collapsing the policy. Non-trivial transfer would require normalizing the observation by environment dimensions or using a function approximator.

Acknowledgements

Code: https://github.com/comegenet/RL-Text_Flappy_Bird

References

1. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press (2018)
2. Christodoulidis, S.: Text Flappy Bird Gym. https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym (2024)
3. Talendar: Flappy Bird Gym. https://github.com/Talendar/flappy-bird-gym (2021)
4. Mnih, V., et al.: Human-level control through deep reinforcement learning. Nature **518**, 529–533 (2015)